

函数的基本概念

在 C++ 中，函数可以理解为 **一段完成特定功能的代码**。

函数基础

例如：

- 计算两个数的最大值
- 判断一个数是否为质数
- 计算 1 到 n 的和

如果这些操作在程序中需要使用很多次，那么每次都重新写一遍代码会非常麻烦。

这时我们就可以把这段代码单独写成一个**函数**，需要时直接调用即可。

例如：

```
1  int add(int a,int b){
2      return a+b;
3  }
```

这个函数的作用就是：接收两个整数 `a`、`b`，并返回它们的和。

函数最大的作用就是：

- **减少重复代码**
- **让程序结构更加清晰**
- **方便调试与维护**

函数的定义

一个普通函数的基本格式如下：

```
1  返回值类型 函数名(参数列表){
2      函数体;
3      return 返回值;
4  }
```

例如：

```
1  int add(int a,int b){
2      return a+b;
3  }
```

其中：

- `int`：返回值类型

- `add` : 函数名
- `int a,int b` : 参数列表
- `{ }` 中的内容: 函数体
- `return a+b;` : 返回结果

函数的调用

函数定义好之后, 需要通过 **函数名** 来调用。

例如:

```
1  int add(int a,int b){
2      return a+b;
3  }
4
5  int main(){
6      int x=3,y=5;
7      cout<<add(x,y);
8      return 0;
9  }
```

输出:

```
1  8
```

函数的参数

函数可以接收外部传入的数据, 这些数据叫做 **参数**。

例如:

```
1  int Max(int a,int b){
2      if(a>b) return a;
3      else return b;
4  }
```

这里的 `a` 和 `b` 就是函数的参数。

调用时:

```
1  cout<<Max(10,7);
```

那么:

```
1  a=10
2  b=7
```

最终返回 `10`。

形参与实参

函数定义时写在参数列表中的变量叫做 **形式参数（形参）**。

例如：

```
1  int add(int a,int b)
```

其中 **a**、**b** 是形参。

函数调用时传入的具体数据叫做 **实际参数（实参）**。

例如：

```
1  add(3,5);
```

其中 **3**、**5** 是实参。

也可以传入变量：

```
1  int x=3,y=5;  
2  add(x,y);
```

此时 **x**、**y** 是实参。

函数的返回值

函数可以通过 **return** 返回一个结果。

例如：

```
1  int square(int x){  
2      return x*x;  
3  }
```

调用：

```
1  cout<<square(5);
```

输出：

```
1  25
```

也可以先把返回值存入变量：

```
1  int ans=square(5);  
2  cout<<ans;
```

return 的作用

`return` 有两个常见作用：

1. 返回函数的结果
2. 立即结束当前函数

例如：

```
1  int f(int x){
2      if(x<0) return -1;
3      return x*x;
4  }
```

如果 `x<0`，执行 `return -1;` 后函数就会立刻结束，后面的代码不会继续执行。

没有返回值的函数

有些函数只需要完成一个操作，不需要返回结果。

这时返回值类型使用 `void`。

例如：

```
1  void hello(){
2      cout<<"Hello World!\n";
3  }
```

调用：

```
1  int main(){
2      hello();
3      return 0;
4  }
```

输出：

```
1  Hello World!
```

`void` 可以理解为：**这个函数没有返回值。**

例题：#Luogu0100.【深基7.例1】距离函数

思路：

根据题意，定义一个求两点之间距离的函数。

返回值类型 `double`。形参：`int x1,int y1,int x2,int y2`。分别表示两个点的横纵坐标。

```
1  @open
2  double dis(double x1,double y1,double x2,double y2){
3      return sqrt(pow(x2-x1,2)+pow(y2-y1,2));
4  }
```

接下来，调用函数，分别求出三边的长度之和即为三角形的周长。

代码：

```
1  @fold
2  #include<bits/stdc++.h>
3
4  using namespace std;
5
6  double dis(double x1,double y1,double x2,double y2){
7      return sqrt(pow(x2-x1,2)+pow(y2-y1,2));
8  }
9
10 int main(){
11     double a,b,c,d,e,f;
12     cin>>a>>b>>c>>d>>e>>f;
13     double ans=dis(a,b,c,d)+dis(a,b,e,f)+dis(c,d,e,f);
14     cout<<fixed<<setprecision(2)<<ans;
15     return 0;
16 }
```

例题： #Luogu0101. 【深基7.例2】质数筛

思路：

需要判断每个数是否是质数，我们可以定义一个判断**质数**的函数。

一个大于 1 的正整数，如果它的正因数只有 1 和它本身，那么称这个数为**质数**，也称为素数。

例如：

2, 3, 5, 7, 11, 13, 17, 19, ...

都是质数。

一个大于 1 的正整数，如果它除了 1 和自身以外还有其他正因数，那么称这个数为**合数**。

例如：

4, 6, 8, 9, 10, 12 等都是合数。

注意：1 既不是质数，也不是合数。

我们可以尝试去寻找是否存在除了 1 和它本身之外的因子。

```
1  @open
2  bool isPrime(int n){
3      if(n<2) return false;
4      for(int i=2;i<n;i++){
5          if(n%i==0) return false;
6      }
7      return true;
8  }
```

这样判断质数需要做大约 n 次判断。容易发现，因子总是成对出现的。即如果 i 是 n 的因子，那么 n/i 也一定是 i 的因子。所以我们只需要枚举到 $\sqrt{n}$ 即可。

```
1  @open 判断质数模板
2  bool isPrime(int n){
3      if(n<2) return false;
4      for(int i=2;i≤n/i;i++){
5          if(n%i==0) return false;
6      }
7      return true;
8  }
```

接下来，调用函数依次判断每个数是否是质数即可。

代码：

```
1  @fold
2  #include<bits/stdc++.h>
3
4  using namespace std;
5
6  bool isPrime(int n){
7      if(n<2) return false;
8      for(int i=2;i≤n/i;i++){
9          if(n%i==0) return false;
10     }
11     return true;
12 }
13
14 int main(){
15     int n;
16     cin>>n;
17     while(n--){
18         int x;
19         cin>>x;
20         if(isPrime(x)) cout<<x<<" ";
21     }
22     return 0;
23 }
```

例题：#Luogu0102. 【深基7.例3】闰年展示

思路：

本题较为简单，需要定义一个判断闰年的函数。

```
1  @open
2  bool check(int y){
3      if((y%4==0&& y%100≠0) || y%400==0) return true;
4      return false;
5  }
```

或者简写为：

```

1  @open
2  bool check(int y){
3      return ((y%4==0&& y%100!=0) || y%400==0);
4  }

```

接下来，依次判断每个年份是否是闰年，存进数组中，并统一输出。

代码：

```

1  @fold
2  #include<bits/stdc++.h>
3
4  using namespace std;
5
6  bool check(int y){
7      return ((y%4==0&& y%100!=0) || y%400==0);
8  }
9
10 int a[3010],cnt;
11 int main(){
12     int x,y;
13     cin>>x>>y;
14     for(int i=x;i<=y;i++){
15         if(check(i)) a[cnt++]=i;
16     }
17
18     cout<<cnt<<"\n";
19     for(int i=0;i<cnt;i++) cout<<a[i]<<" ";
20     return 0;
21 }

```